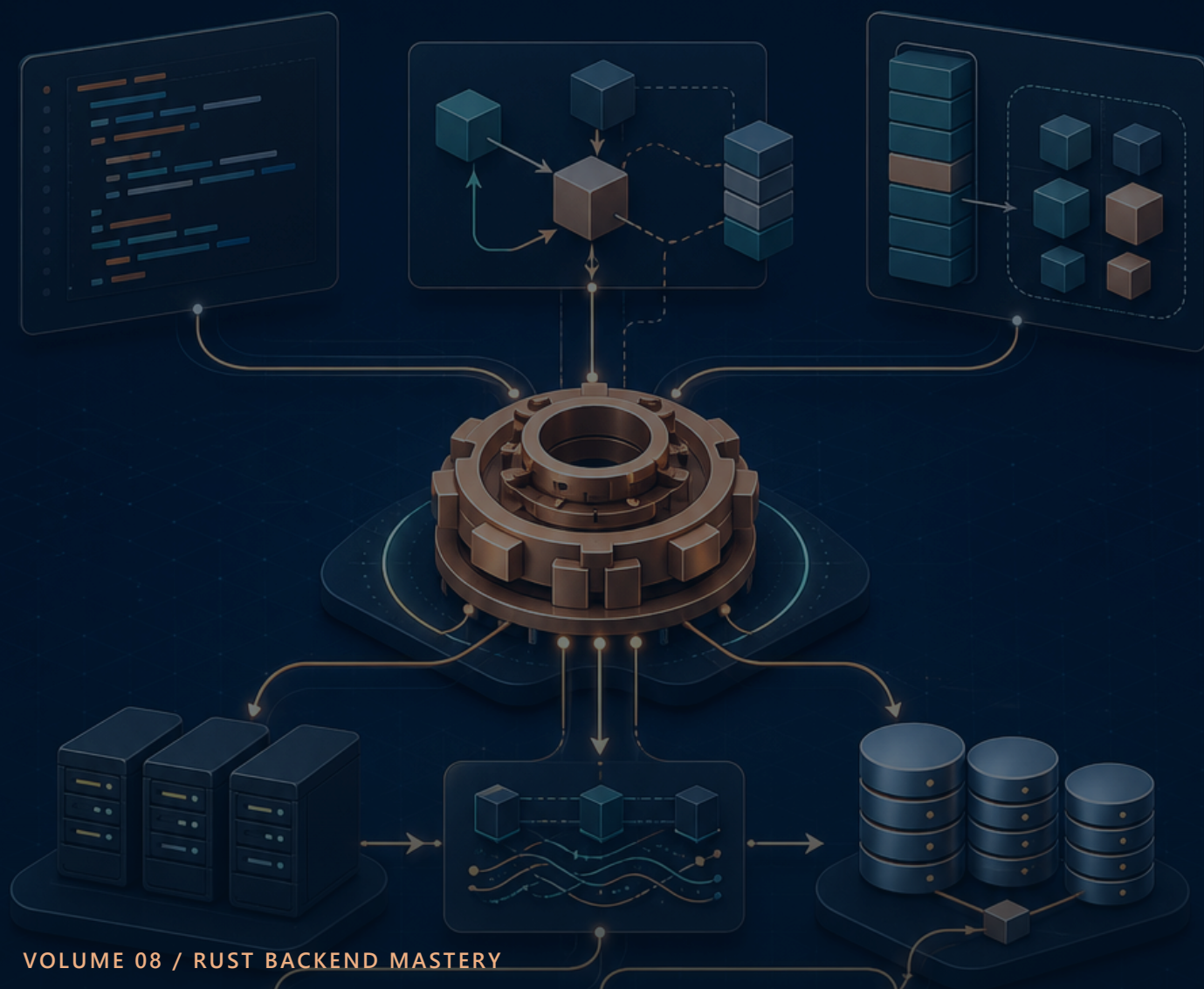




VOLUME 08 / RUST BACKEND MASTERY

認証・認可 — Topcoat Session・Argon2id・Better Auth比較

passwordからsession lifecycle、CSRF、IDOR、OIDC、incidentまでを脅威モデルから設計する

10

深掘り単元

20+

実行・解答例

40+

図・表・画面

実務

設計と障害対応

LEARNING MAP

この巻で身に付けること

Better Authを無条件に採用せず、Rust単体構成・TypeScript sidecar・managed IdPを要件と運用境界で比較します。

到達目標

- ✓ Argon2idとsessionを安全に実装できる
- ✓ CSRF・IDOR・列挙をテストできる
- ✓ Better Auth sidecarの増える境界を説明できる
- ✓ OIDCとincident対応の責任範囲を理解する

学習地図

01 認証・認可・session・脅威モデル

認証は主体、sessionは継続、認可は操作許可を決めます。守る資産、攻撃者、入口、信頼境界を図にし、資格情報漏えい、session hijack、CSRF、IDORを先に列挙します。

02 パスワード・Argon2id・salt

パスワードは暗号化ではなく、ランダムsalt付きの遅いpassword hashへ変換します。Argon2idのPHC文字列にalgorithm、parameter、salt、hashを保存し、平文や独自salt列を保存しません。

03 登録・email正規化・重複・列挙対策

登録ではemailをtrim・小文字化し、password policyを検証してからhashします。DBのUNIQUE制約を最終判断にし、競合を409または一般化した表示へ変換します。

04 login・generic error・rate limit

loginではemail存在の有無とpassword誤りを同じ外部メッセージにし、内部ログだけ分類します。成功・失敗とも時間差を小さくし、IPとaccountの両軸でrate limitします。

05 Topcoat session lifecycle

Topcoat session::startは新tokenをcookieへ発行し、保存用token_hashとexpires_atを返します。token_hashをDBへ保存し、tokenそのものは保存しません。stop、refresh、rotateもDB操作と対にします。

06 cookie・CSRF・Origin・SameSite

session cookieはSecure、HttpOnly、SameSite、Pathを硬化します。しかしSameSiteだけで全CSRFを防げないため、Topcoat .sessionsのOriginLayerを維持し、状態変更はGETにしません。

07 認可・IDOR・tenant境界

URLのnote_idを受け取っても、それがcurrent userの所有物を必ず確認します。SELECT/UPDATE/DELETEのWHEREへuser_idを含め、存在と所有を外部へ区別しすぎません。

08 Better Auth比較と推奨構成

Better AuthはTypeScriptライブラリで、libSQL対応のrelational adapterを持ちますが、Topcoat内へ直接組み込むnative Rust crateではありません。導入にはTS auth serviceとcookie/domain契約が増えます。

09 OIDC・managed IdP・MFA・回復

企業SSOやsocial login、MFA、account recoveryが主要要件なら、自作資格情報よりOIDC対応IdPを検討します。Authorization Code + PKCE、state、nonce、redirect URIを守ります。

10 security test・監査・incident response

認証は正常loginだけでなく、列挙、CSRF、session fixation、期限、logout後、他人resource、rate limit、秘密漏えいを自動テストします。監査eventは誰が何をいつ試みたかを残します。

HOW TO USE THIS VOLUME

読むだけで終わらせない進め方

「予想→実行→観察→一か所変更→回帰テスト」を一单元ごとに繰り返します。写経した行数ではなく、入力・処理・出力・失敗を自分の言葉で説明できるかを進捗にします。

1. 予想

実行前に入力型、出力、失敗箇所を予想します。

2. そのまま実行

最初は変更せず、教材と同じ出力が比較します。

3. 一か所変更

値または型を一つだけ変え、差分の理由を説明します。

4. 回帰テスト

直した失敗を自動テストへ残し、再発を防ぎます。

STEP 1

AuthNとAuthZ

STEP 2

資産と攻撃者

STEP 3

信頼境界

STEP 4

失敗時の既定拒否

毎回そろえる確認コマンド

● ● ● 単元を終える前の品質確認

```
cargo fmt --check
cargo clippy --all-targets -- -D warnings
cargo test --locked --all-targets
```

対象	この教材の基準	混在を防ぐ理由
Rust	2024 Edition / rust-version 1.95以上	editionとMSRVを明示し、環境差を再現可能にする
Topcoat	crate / CLI ともに 0.5.0固定	early-stageのため、mainブランチの別構文を混ぜない
DB・認証	Turso/libSQL、Argon2id、Topcoat Session	一つのRustプロセスで責務とデータの流れを追える
公開	Docker + Fly.io、HOST=0.0.0.0、PORT=8080	ローカルとコンテナの待受差を明示する

重要な版の前提 Topcoatは公式がearly-stage / experimentalと明記しています。本教材はcrateとCLIを **0.5.0** に固定し、mainブランチの別構文を混ぜません。

UNIT 01

認証・認可・session・脅威モデル

UNIT 01・CONCEPT

認証・認可・session・脅威モデル

認証は主体、sessionは継続、認可は操作許可を決めます。守る資産、攻撃者、入口、信頼境界を図にし、資格情報漏えい、session hijack、CSRF、IDORを先に列挙します。

この単元の到達点

- AuthNとAuthZ
- 資産と攻撃者
- 信頼境界
- 失敗時の既定拒否

なぜバックエンドで必要か

login済みかだけでなく、routeごと・resourceごとにactorとownerを照合します。SQLのWHEREにもuser_idを含めます。



UNIT 01・MENTAL MODEL

認証・認可・session・脅威モデルを図でつかむ



認証は主体、sessionは継続、認可は操作許可を決めます。守る資産、攻撃者、入口、信頼境界を図にし、資格情報漏えい、session hijack、CSRF、IDORを先に列挙します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	AuthNとAuthZ	AuthNとAuthZ — 入力と前提を確認し、境界を曖昧にしない。
2	資産と攻撃者	資産と攻撃者 — 値がどの順で変換されるかを追跡する。
3	信頼境界	信頼境界 — 失敗を再現し、型またはログから原因を特定する。
4	失敗時の既定拒否	失敗時の既定拒否 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

認証・認可・session・脅威モデルとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 01・MINIMUM CODE

まず動く最小コード

認証・認可・session・脅威モデルだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
struct Actor{user_id:i64}
struct Note{owner_id:i64}
fn authorize(actor:&Actor,note:&Note)->Result<(),AuthzError>{if
actor.user_id==note.owner_id{Ok(())}else{Err(AuthzError::Forbidden)}}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 01・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>struct Actor{user_id:i64}</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>struct Note{owner_id:i64}</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn authorize(actor:&Actor,note:&Note)->Result<(),AuthzError>{if actor.user</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。

実行時に起きる順序

- ✓ 入力AuthNとAuthZの条件を満たすか確認する。
- ✓ 資産と攻撃者を実行し、途中の型と状態を追う。
- ✓ 信頼境界で失敗を成功経路から分離する。
- ✓ 失敗時の既定拒否により、出力と副作用を検証する。

型を声に出す 認証・認可・session・脅威モデルに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 01・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

owner一致 → Ok
不一致 → Forbidden

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 01・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

資産: password hash/session token/note
 攻撃者: 未認証者/別user/漏えいtoken所持者
 入口: register/login/CRUD/procedure
 対策: Argon2id/hashed session/CSRF/owner SQL/rate limit
 失敗既定: deny + generic response

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 01・BACKEND APPLICATION

バックエンドのどこで使うか

login済みかだけでなく、routeごと・resourceごとにactorとownerを照合します。SQLのWHEREにもuser_idを含めます。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 01・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	AuthNとAuthZに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	資産と攻撃者の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	信頼境界または失敗時の既定拒否の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 01・DEBUGGING

認証・認可・session・脅威モデルを再現して切り分ける

認証・認可・session・脅威モデルの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: AuthNとAuthZ
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 資産と攻撃者
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 信頼境界
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 失敗時の既定拒否

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	AuthNとAuthZ	AuthNとAuthZ — 入力と前提を確認し、境界を曖昧にしない。
2	資産と攻撃者	資産と攻撃者 — 値がどの順で変換されるかを追跡する。
3	信頼境界	信頼境界 — 失敗を再現し、型またはログから原因を特定する。
4	失敗時の既定拒否	失敗時の既定拒否 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 認証後も毎操作で認可し、曖昧な場合は拒否します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 01 · PRACTICE

手を動かす演習

RustNoteの資産、攻撃者、入口、最悪影響、対策を5行の脅威モデルにしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 01・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

資産: password hash/session token/note

攻撃者: 未認証者/別user/漏えいtoken所持者

入口: register/login/CRUD/procedure

対策: Argon2id/hashed session/CSRF/owner SQL/rate limit

失敗既定: deny + generic response

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 認証後も毎操作で認可し、曖昧な場合は拒否します。
- ✓ 認証・認可・session・脅威モデルとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 02

パスワード・Argon2id・salt

UNIT 02 · CONCEPT

パスワード・Argon2id・salt

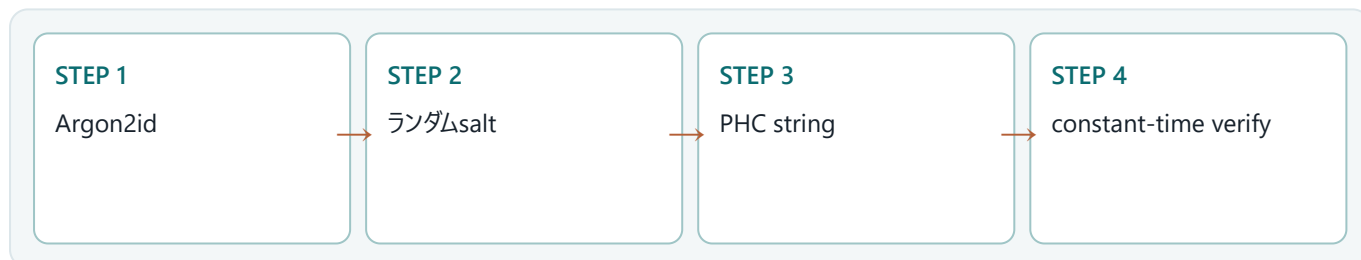
パスワードは暗号化ではなく、ランダムsalt付きの遅いpassword hashへ変換します。Argon2idのPHC文字列にalgorithm、parameter、salt、hashを保存し、平文や独自salt列を保存しません。

この単元の到達点

- Argon2id
- ランダムsalt
- PHC string
- constant-time verify

なぜバックエンドで必要か

hash処理はspawn_blockingへ隔離し、同時数を制限します。parameterは負荷試験で調整し、将来はlogin成功時のrehashを検討します。



先に答え

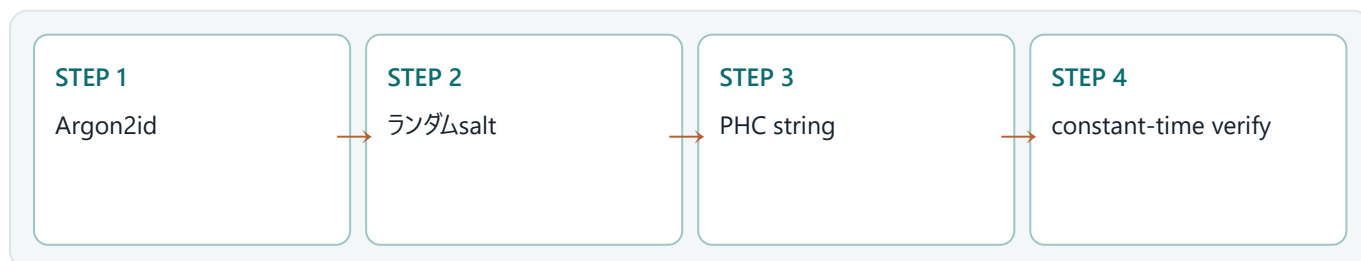
平文ではなくsalt付きArgon2id PHC文字列を保存し、検証はpassword APIへ任せます。

確認する問い

パスワード・Argon2id・saltを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 02 · MENTAL MODEL

パスワード・Argon2id・saltを図でつかむ



パスワードは暗号化ではなく、ランダムsalt付きの遅いpassword hashへ変換します。Argon2idのPHC文字列にalgorithm、parameter、salt、hashを保存し、平文や独自salt列を保存しません。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では

結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	Argon2id	Argon2id — 入力と前提を確認し、境界を曖昧にしない。
2	ランダムsalt	ランダムsalt — 値がどの順で変換されるかを追跡する。
3	PHC string	PHC string — 失敗を再現し、型またはログから原因を特定する。
4	constant-time verify	constant-time verify — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

パスワード・Argon2id・saltとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 02 · MINIMUM CODE

まず動く最小コード

パスワード・Argon2id・saltだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
use argon2::{Argon2, PasswordHash, PasswordHasher, PasswordVerifier}; use argon2::password_hash::{
    SaltString, rand_core::OsRng};
let salt=SaltString::generate(&mut OsRng); let
hash=Argon2::default().hash_password(password.as_bytes(),&salt)?.to_string();
let parsed=PasswordHash::new(&hash)?.Argon2::default().verify_password(candidate.as_bytes(),&parsed)?;
```

● ● ● 実行コマンド

```
cargo test password
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 02 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>use argon2::{Argon2, PasswordHash, PasswordHasher, PasswordVerifier}; use argo</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>let salt = SaltString::generate(&mut OsRng); let hash = Argon2::default().hash_</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let parsed = PasswordHash::new(&hash)?; Argon2::default().verify_password(can</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。

実行時に起きる順序

- ✓ 入力がArgon2idの条件を満たすか確認する。
- ✓ ランダムsaltを実行し、途中の型と状態を追う。
- ✓ PHC stringで失敗を成功経路から分離する。
- ✓ constant-time verifyにより、出力と副作用を検証する。

型を声に出す パスワード・Argon2id・saltに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 02 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
hash=$argon2id$v=19$m=...$<salt>$<digest>
verify correct → Ok
verify wrong → Err
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 02 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[test]fn salted_hashes_differ(){let a=hash_password("secret").unwrap();let b=hash_password("secret").unwrap();assert_ne!(a,b);assert!(verify_password("secret",&a));assert!(verify_password("secret",&b));}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

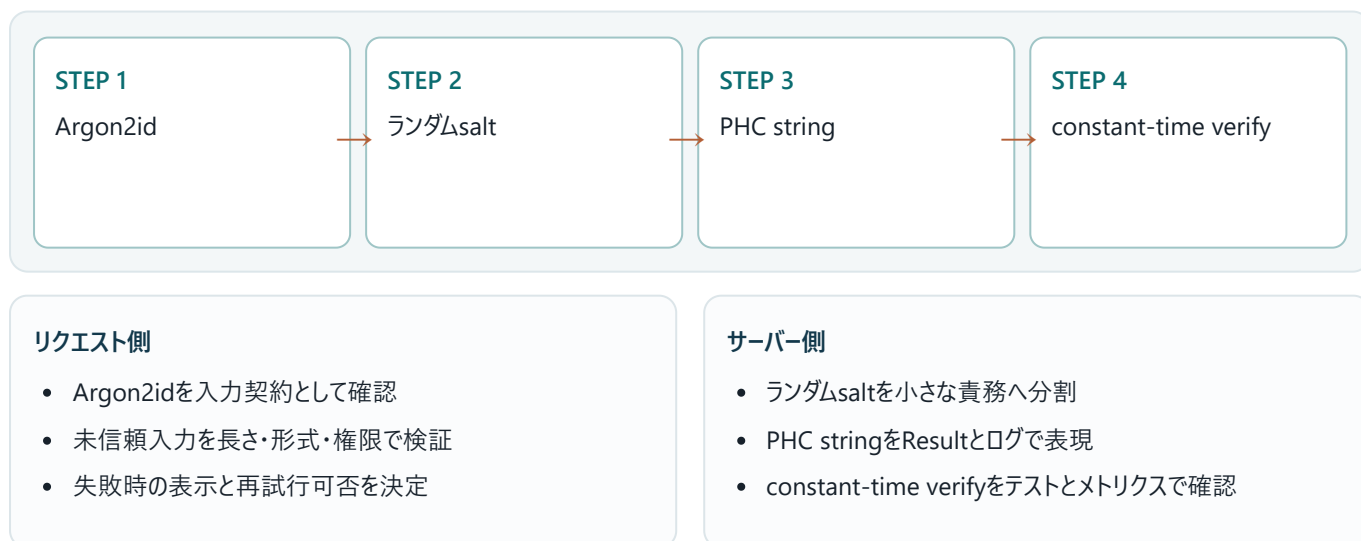
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 02 · BACKEND APPLICATION

バックエンドのどこで使うか

hash処理はspawn_blockingへ隔離し、同時数を制限します。parameterは負荷試験で調整し、将来はlogin成功時のrehashを検討します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。

- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 02 ・ FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	Argon2idに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	ランダムsaltの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	PHC stringまたはconstant-time verifyの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 02・DEBUGGING

パスワード・Argon2id・saltを再現して切り分ける

パスワード・Argon2id・saltの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: Argon2id
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: ランダムsalt
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: PHC string
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: constant-time verify

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	Argon2id	Argon2id — 入力と前提を確認し、境界を曖昧にしない。
2	ランダムsalt	ランダムsalt — 値がどの順で変換されるかを追跡する。
3	PHC string	PHC string — 失敗を再現し、型またはログから原因を特定する。
4	constant-time verify	constant-time verify — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 平文ではなくsalt付きArgon2id PHC文字列を保存し、検証はpassword APIへ任せます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 02 ・ PRACTICE

手を動かす演習

同じpasswordを二回hashして異なる文字列になるが、両方verifyできるテストを書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 02 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[test]fn salted_hashes_differ(){let a=hash_password("secret").unwrap();let b=hash_password("secret").unwrap();assert_ne!(a,b);assert!(verify_password("secret",&a));assert!(verify_password("secret",&b));}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 平文ではなくsalt付きArgon2id PHC文字列を保存し、検証はpassword APIへ任せます。
- ✓ パスワード・Argon2id・saltとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 03

登録・email正規化・重複・列挙対策

UNIT 03 · CONCEPT

登録・email正規化・重複・列挙対策

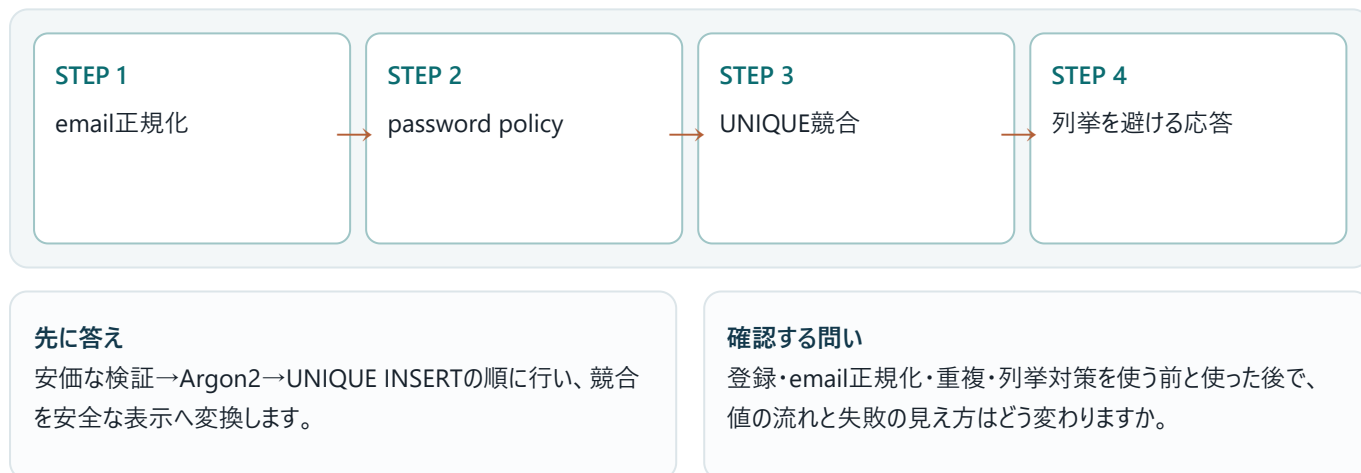
登録ではemailをtrim・小文字化し、password policyを検証してからhashします。DBのUNIQUE制約を最終判断にし、競合を409または一般化した表示へ変換します。

この単元の到達点

- email正規化
- password policy
- UNIQUE競合
- 列挙を避ける応答

なぜバックエンドで必要か

登録と同時にsessionを開始するか、email verification後に開始するかを要件で決めます。重いhashより前に安価な形式検証を行います。



UNIT 03 · MENTAL MODEL

登録・email正規化・重複・列挙対策を図でつかむ



登録ではemailをtrim・小文字化し、password policyを検証してからhashします。DBのUNIQUE制約を最終判断にし、競合を409または一般化した表示へ変換します。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	email正規化	email正規化 — 入力と前提を確認し、境界を曖昧にしない。
2	password policy	password policy — 値がどの順で変換されるかを追跡する。
3	UNIQUE競合	UNIQUE競合 — 失敗を再現し、型またはログから原因を特定する。
4	列挙を避ける応答	列挙を避ける応答 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

登録・email正規化・重複・列挙対策とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 03 · MINIMUM CODE

まず動く最小コード

登録・email正規化・重複・列挙対策だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
fn normalize_email(raw:&str)->Result<String,&'static str>{let e=raw.trim().to_lowercase();if e.len()
<=254&&e.contains('@'){Ok(e)}else{Err("invalid email")}}
// INSERT users(email,password_hash) VALUES(?1,?2)
// UNIQUE violationは既存emailを過度に詳しく漏らさない応答へ変換
```

● ● ● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 03 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>fn normalize_email(raw:&str)->Result<String,&'static str>{let e=raw.trim()</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
登録・email正規化・重複・列挙対策	入力、処理、出力を分離して読みます。
登録・email正規化・重複・列挙対策	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がemail正規化の条件を満たすか確認する。
- ✓ password policyを実行し、途中の型と状態を追う。
- ✓ UNIQUE競合で失敗を成功経路から分離する。
- ✓ 列挙を避ける応答により、出力と副作用を検証する。

型を声に出す 登録・email正規化・重複・列挙対策に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 03・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

登録・email正規化・重複・列挙対策

入力値は保持され、emailとpasswordの近くに具体的な修正メッセージが表示されます。password値だけは再表示しません。

観察ポイント

- email正規化
- password policy
- UNIQUE競合

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 03・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[derive(Default, Debug)] struct RegisterErrors { email: Option<&'static str>, password: Option<&'static str> } fn validate(e: &str, p: &str) -> Result<(), RegisterErrors> { let mut x = RegisterErrors::default(); if !e.contains('@') { x.email = Some("invalid"); } if p.chars().count() < 12 { x.password = Some("12+ chars"); } if x.email.is_some() || x.password.is_some() { Err(x) } else { Ok(()) } }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 03・BACKEND APPLICATION

バックエンドのどこで使うか

登録と同時にsessionを開始するか、email verification後に開始するかを要件で決めます。重いhashより前に安価な形式検証を行います。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 03 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	email正規化に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	password policyの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	UNIQUE競合または列挙を避ける応答の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 03・DEBUGGING

登録・email正規化・重複・列挙対策を再現して切り分ける

登録・email正規化・重複・列挙対策の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: email正規化
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: password policy
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: UNIQUE競合
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 列挙を避ける応答

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	email正規化	email正規化 — 入力と前提を確認し、境界を曖昧にしない。
2	password policy	password policy — 値がどの順で変換されるかを追跡する。
3	UNIQUE競合	UNIQUE競合 — 失敗を再現し、型またはログから原因を特定する。
4	列挙を避ける応答	列挙を避ける応答 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 安価な検証→Argon2→UNIQUE INSERTの順に行い、競合を安全な表示へ変換します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 03 ・ PRACTICE

手を動かす演習

emailとpasswordの項目別エラーを返すRegisterErrors型を作ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 03 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[derive(Default, Debug)]struct RegisterErrors{email:Option<&'static str>,password:Option<&'static str>}fn validate(e:&str,p:&str)->Result<(),RegisterErrors>{let mut x=RegisterErrors::default();if !e.contains('@'){x.email=Some("invalid")}if p.chars().count(<12){x.password=Some("12+ chars")}if x.email.is_some()||x.password.is_some(){Err(x)}else{Ok(())}}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 安価な検証→Argon2→UNIQUE INSERTの順に行い、競合を安全な表示へ変換します。
- ✓ 登録・email正規化・重複・列挙対策とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 04

login・generic error・rate limit

UNIT 04・CONCEPT

login・generic error・rate limit

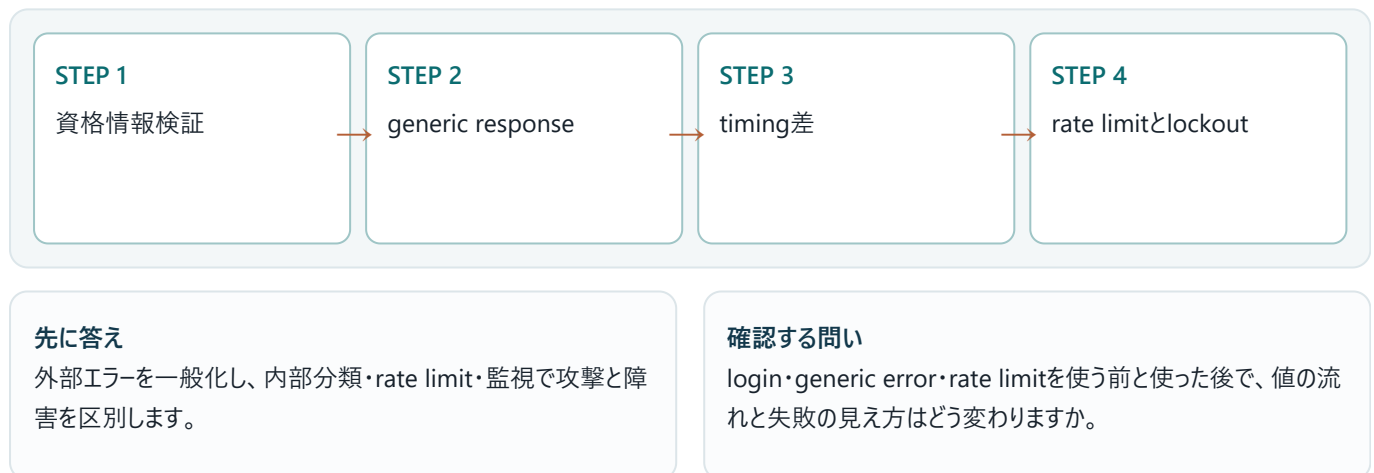
loginではemail存在の有無とpassword誤りを同じ外部メッセージにし、内部ログだけ分類します。成功・失敗とも時間差を小さくし、IPとaccountの両軸でrate limitします。

この単元の到達点

- 資格情報検証
- generic response
- timing差
- rate limitとlockout

なぜバックエンドで必要か

固定lockoutは攻撃者が他人を締め出せるため、遅延、短期bucket、CAPTCHA/step-up、通知を組み合わせます。



UNIT 04・MENTAL MODEL

login・generic error・rate limitを図でつかむ



loginではemail存在の有無とpassword誤りを同じ外部メッセージにし、内部ログだけ分類します。成功・失敗とも時間差を小さくし、IPとaccountの両軸でrate limitします。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	資格情報検証	資格情報検証 — 入力と前提を確認し、境界を曖昧にしない。
2	generic response	generic response — 値がどの順で変換されるかを追跡する。
3	timing差	timing差 — 失敗を再現し、型またはログから原因を特定する。
4	rate limitとlockout	rate limitとlockout — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

login・generic error・rate limitとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 04 · MINIMUM CODE

まず動く最小コード

login・generic error・rate limitだけに注目した最小例です。まず変更せず実行し、出力が一致した後に一行ずつ意味を確認します。

```
async fn login(input: Login) -> Result<User, LoginError> {
    let row = find_user(&input.email).await?;
    let hash = row.as_ref().map(|u| u.password_hash.as_str()).unwrap_or(DUMMY_HASH);
    let valid = verify_blocking(&input.password, hash).await?;
    match (row, valid) {
        (Some(user), true) => Ok(user),
        _ => Err(LoginError::InvalidCredentials)}
}
```

● ● ● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 04 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>async fn login(input:Login)->Result<User,LoginError>{let row=find_user(&in</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>login・generic error・rate limit</code>	入力、処理、出力を分離して読みます。
<code>login・generic error・rate limit</code>	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力資格情報検証の条件を満たすか確認する。
- ✓ generic responseを実行し、途中の型と状態を追う。
- ✓ timing差で失敗を成功経路から分離する。
- ✓ rate limitとlockoutにより、出力と副作用を検証する。

型を声に出す login・generic error・rate limitに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 04 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

失敗: emailまたはパスワードが正しくありません
内部: reason=user_missing または password_mismatch

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 04 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。


```
#[derive(Default)]struct Attempts{failures:u8}impl Attempts{fn fail(&mut self)->bool{self.failures=self.failures.saturating_add(1);self.failures<=10}fn success(&mut self){self.failures=0}}fn main(){let mut a=Attempts::default();for _ in 0..10{assert!(a.fail())}assert!(a.fail());a.success();}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

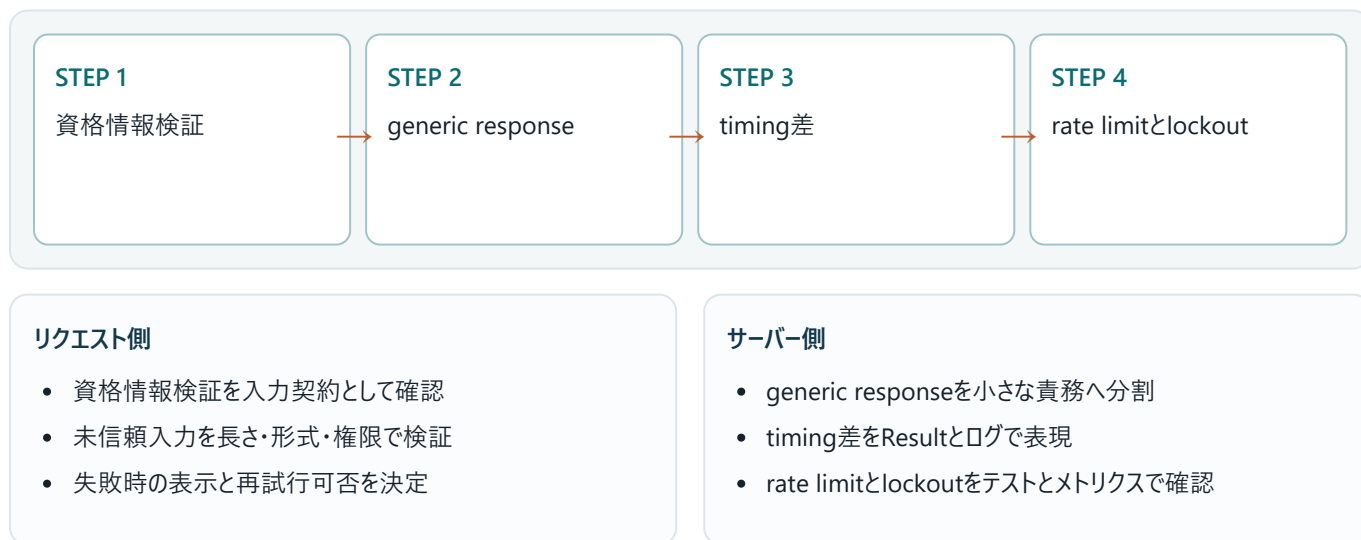
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 04 · BACKEND APPLICATION

バックエンドのどこで使うか

固定lockoutは攻撃者が他人を締め出せるため、遅延、短期bucket、CAPTCHA/step-up、通知を組み合わせます。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 04 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	資格情報検証に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	generic responseの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	timing差またはrate limitとlockoutの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 04・DEBUGGING

login・generic error・rate limitを再現して切り分ける

login・generic error・rate limitの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 資格情報検証
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: generic response
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: timing差
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: rate limitとlockout

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	資格情報検証	資格情報検証 — 入力と前提を確認し、境界を曖昧にしない。
2	generic response	generic response — 値がどの順で変換されるかを追跡する。
3	timing差	timing差 — 失敗を再現し、型またはログから原因を特定する。
4	rate limitとlockout	rate limitとlockout — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 外部エラーを一般化し、内部分類・rate limit・監視で攻撃と障害を区別します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 04 · PRACTICE

手を動かす演習

5分10回のtoken bucket判定を純粋関数で表し、成功時に失敗回数をresetしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 04 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[derive(Default)]struct Attempts{failures:u8}impl Attempts{fn fail(&mut self)->bool{self.failures=self.failures.saturating_add(1);self.failures<=10}fn success(&mut self){self.failures=0}}fn main(){let mut a=Attempts::default();for _ in 0..10{assert!(a.fail())}assert!(a.fail());a.success();}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 外部エラーを一般化し、内部分類・rate limit・監視で攻撃と障害を区別します。
- ✓ login・generic error・rate limitとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 05

Topcoat session lifecycle

UNIT 05 · CONCEPT

Topcoat session lifecycle

Topcoat session::startは新tokenをcookieへ発行し、保存用token_hashとexpires_atを返します。token_hashをDBへ保存し、tokenそのものは保存しません。stop、refresh、rotateもDB操作と対にします。

この単元の到達点

- startとhash保存
- token_hash lookup
- stopと削除
- refresh・rotate

なぜバックエンドで必要か

privilege変更ではrotateし、古いhash削除と新hash追加をtransactionで行います。全端末logoutはuser_idの全session行を削除します。



先に答え

cookie token、DB hash、期限、userを一つのlifecycleとして更新します。

確認する問い

Topcoat session lifecycleを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 05 · MENTAL MODEL

Topcoat session lifecycleを図でつかむ



Topcoat session::startは新tokenをcookieへ発行し、保存用token_hashとexpires_atを返します。token_hashをDBへ保存し、tokenそのものは保存しません。stop、refresh、rotateもDB操作と対にします。図では左から右へ処理を追います。各箱の入口では入力型と信頼

度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	startとhash保存	startとhash保存 — 入力と前提を確認し、境界を曖昧にしない。
2	token_hash lookup	token_hash lookup — 値がどの順で変換されるかを追跡する。
3	stopと削除	stopと削除 — 失敗を再現し、型またはログから原因を特定する。
4	refresh・rotate	refresh・rotate — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

Topcoat session lifecycleとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 05 ・ MINIMUM CODE

まず動く最小コード

Topcoat session lifecycleだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
let
session=session::start(cx).await?;repo.insert_session(user.id,&session.token_hash,session.expires_at).await?
;
// request
let Some(hash)=session::token_hash(cx).await? else{return Ok(None)};repo.find_unexpired_user(&hash).await
// logout
if let Some(hash)=session::stop(cx).await?{repo.delete_session(&hash).await?;}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 05 ・ LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>let session=session::start(cx).await?;repo.insert_session(user.id,&session</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let Some(hash)=session::token_hash(cx).await? else{return Ok(None)};repo.f</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>if let Some(hash)=session::stop(cx).await? {repo.delete_session(&hash).awai</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。

実行時に起きる順序

- ✓ 入力 start と hash 保存の条件を満たすか確認する。
- ✓ token_hash lookup を実行し、途中の型と状態を追う。
- ✓ stop と削除で失敗を成功経路から分離する。
- ✓ refresh・rotate により、出力と副作用を検証する。

型を声に出す Topcoat session lifecycle に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 05・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

login → __Host-... Secure HttpOnly cookie + DB hash
logout → cookie破棄 + DB row削除

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 05 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
async fn current_user(cx:&Cx,repo:&Repo)->Result<Option<User>>{let Some(hash)=session::token_hash(cx).await?
else{return Ok(None)};match repo.find_session(&hash).await?{Some(s) if
s.expires_at>now()->Ok(Some(s.user)),Some(_)->{repo.delete_session(&hash).await?;Ok(None)},None=>Ok(None)}}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 05 ・ BACKEND APPLICATION

バックエンドのどこで使うか

privilege変更ではrotateし、古いhash削除と新hash追加をtransactionで行います。全端末logoutはuser_idの全session行を削除します。



境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 05 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	startとhash保存に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	token_hash lookupの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	stopと削除またはrefresh・rotateの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 05・DEBUGGING

Topcoat session lifecycleを再現して切り分ける

Topcoat session lifecycleの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: startとhash保存
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: token_hash lookup
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: stopと削除
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: refresh・rotate

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	startとhash保存	startとhash保存 — 入力と前提を確認し、境界を曖昧にしない。
2	token_hash lookup	token_hash lookup — 値がどの順で変換されるかを追跡する。
3	stopと削除	stopと削除 — 失敗を再現し、型またはログから原因を特定する。
4	refresh・rotate	refresh・rotate — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 cookie token、DB hash、期限、userを一つのlifecycleとして更新します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 05 ・ PRACTICE

手を動かす演習

期限切れsessionを無効として返し、DBから削除するcurrent_userの流れを書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 05 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
async fn current_user(cx:&Cx,repo:&Repo)->Result<Option<User>>{let Some(hash)=session::token_hash(cx).await?
else{return Ok(None)};match repo.find_session(&hash).await?{Some(s) if
s.expires_at>now()->Ok(Some(s.user)),Some(_)->{repo.delete_session(&hash).await?;Ok(None)},None=>Ok(None)}}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ cookie token、DB hash、期限、userを一つのlifecycleとして更新します。
- ✓ Topcoat session lifecycleとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 06

cookie・CSRF・Origin・SameSite

UNIT 06 · CONCEPT

cookie・CSRF・Origin・SameSite

session cookieはSecure、HttpOnly、SameSite、Pathを硬化します。しかしSameSiteだけで全CSRFを防げないため、Topcoat .sessionsのOriginLayerを維持し、状態変更はGETにしません。

この単元の到達点

- Secure/HttpOnly
- SameSite
- Origin検証
- 安全なmethod

なぜバックエンドで必要か

dangerous_disable_origin_verificationは独自CSRF対策を完全実装した場合以外使いません。API token clientとbrowser cookie clientの脅威を分けます。



UNIT 06 · MENTAL MODEL

cookie・CSRF・Origin・SameSiteを図でつかむ



session cookieはSecure、HttpOnly、SameSite、Pathを硬化します。しかしSameSiteだけで全CSRFを防げないため、Topcoat .sessionsのOriginLayerを維持し、状態変更はGETにしません。図では左から右へ処理を追います。各箱の入口では入力の種類と信頼度、出口では結

果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	Secure/HttpOnly	Secure/HttpOnly — 入力と前提を確認し、境界を曖昧にしない。
2	SameSite	SameSite — 値がどの順で変換されるかを追跡する。
3	Origin検証	Origin検証 — 失敗を再現し、型またはログから原因を特定する。
4	安全なmethod	安全なmethod — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

cookie・CSRF・Origin・SameSiteとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 06・MINIMUM CODE

まず動く最小コード

cookie・CSRF・Origin・SameSiteだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
let router=Router::builder().cookies().sessions(SessionConfig::default()).build();
// POST/PUT/PATCH/DELETEはSec-Fetch-SiteまたはOriginを検証
// OAuth form_postなど正当なcross-originだけtrust_originで限定許可
```

●●● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 06・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
let router=Router::builder().cookies().sessions(SessionConfig::default()).	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
cookie・CSRF・Origin・SameSite	入力、処理、出力を分離して読みます。
cookie・CSRF・Origin・SameSite	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がSecure/HttpOnlyの条件を満たすか確認する。
- ✓ SameSiteを実行し、途中の型と状態を追う。
- ✓ Origin検証で失敗を成功経路から分離する。
- ✓ 安全なmethodにより、出力と副作用を検証する。

型を声に出す cookie・CSRF・Origin・SameSiteに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 06 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
cross-site POST + session cookie → 403 Forbidden
same-origin POST → handlerへ進む
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 06 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。


```
#[test]fn state_change_must_not_be_get(){let route_method="POST";assert_ne!(route_method,"GET");}
// integration: Origin:https://evil.example + Cookie + Cookie → 403
// Origin:https://app.example + Cookie → handler result
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 06 · BACKEND APPLICATION

バックエンドのどこで使うか

dangerous_disable_origin_verificationは独自CSRF対策を完全実装した場合以外使いません。API token clientとbrowser cookie clientの脅威を分けます。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 06・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	Secure/HttpOnlyに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	SameSiteの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	Origin検証または安全なmethodの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 06・DEBUGGING

cookie・CSRF・Origin・SameSiteを再現して切り分ける

cookie・CSRF・Origin・SameSiteの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: Secure/HttpOnly
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: SameSite
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: Origin検証
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 安全なmethod

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	Secure/HttpOnly	Secure/HttpOnly — 入力と前提を確認し、境界を曖昧にしない。
2	SameSite	SameSite — 値がどの順で変換されるかを追跡する。
3	Origin検証	Origin検証 — 失敗を再現し、型またはログから原因を特定する。
4	安全なmethod	安全なmethod — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 cookie属性、Origin検証、安全なHTTP methodを重ねてCSRFを防ぎます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 06 ・ PRACTICE

手を動かす演習

GET /notes/{id}/deleteをPOST /notes/{id}/deleteへ変更し、origin検証が働くテストケースを書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 06 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[test]fn state_change_must_not_be_get(){let route_method="POST";assert_ne!(route_method,"GET");}
// integration: Origin:https://evil.example + Cookie → 403
// Origin:https://app.example + Cookie → handler result
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ cookie属性、Origin検証、安全なHTTP methodを重ねてCSRFを防ぎます。
- ✓ cookie・CSRF・Origin・SameSiteとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 07

認可・IDOR・tenant境界

UNIT 07 · CONCEPT

認可・IDOR・tenant境界

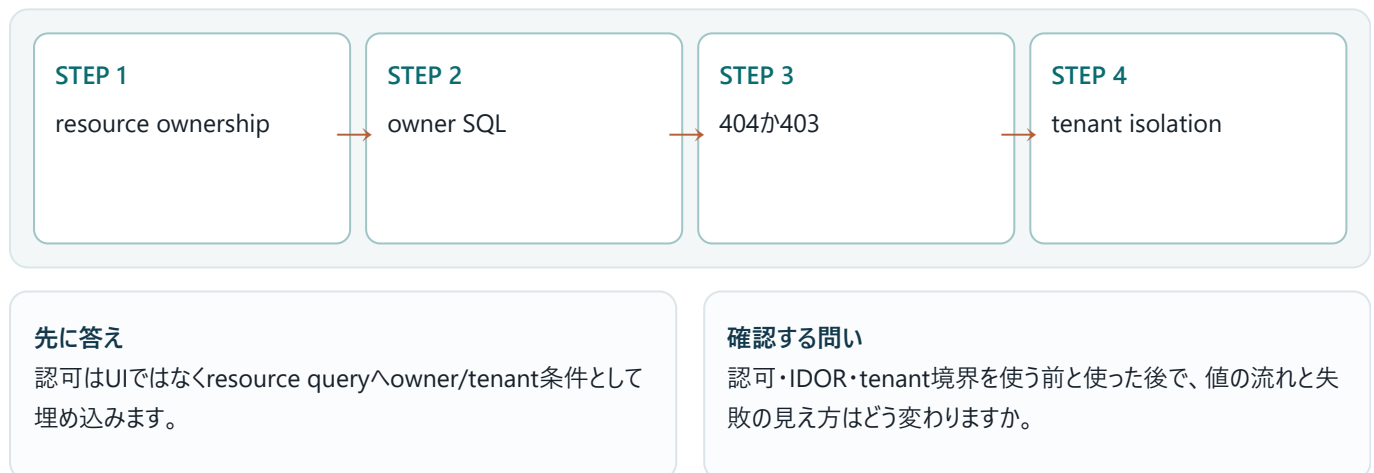
URLのnote_idを受け取っても、それがcurrent userの所有物かを必ず確認します。SELECT/UPDATE/DELETEのWHEREへuser_idを含め、存在と所有を外部へ区別しすぎません。

この単元の到達点

- resource ownership
- owner SQL
- 404か403
- tenant isolation

なぜバックエンドで必要か

管理者操作は暗黙の例外にせずRole/Permission型と監査ログを通します。tenant_idもすべてのqueryとunique indexへ含めます。



UNIT 07 · MENTAL MODEL

認可・IDOR・tenant境界を図でつかむ



URLのnote_idを受け取っても、それがcurrent userの所有物かを必ず確認します。SELECT/UPDATE/DELETEのWHEREへuser_idを含め、存在と所有を外部へ区別しすぎません。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	resource ownership	resource ownership — 入力と前提を確認し、境界を曖昧にしない。
2	owner SQL	owner SQL — 値がどの順で変換されるかを追跡する。
3	404か403	404か403 — 失敗を再現し、型またはログから原因を特定する。
4	tenant isolation	tenant isolation — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

認可・IDOR・tenant境界とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 07 · MINIMUM CODE まず動く最小コード

認可・IDOR・tenant境界だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
UPDATE notes SET title=?1 WHERE id=?2 AND user_id=?3;  
DELETE FROM notes WHERE id=?1 AND user_id=?2;  
SELECT id,title FROM notes WHERE id=?1 AND user_id=?2;
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 07 · LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>UPDATE notes SET title=?1 WHERE id=?2 AND user_id=?3;</code>	失敗時は早期returnし、呼び出し元へエラーを伝播します。
<code>DELETE FROM notes WHERE id=?1 AND user_id=?2;</code>	失敗時は早期returnし、呼び出し元へエラーを伝播します。
<code>SELECT id,title FROM notes WHERE id=?1 AND user_id=?2;</code>	失敗時は早期returnし、呼び出し元へエラーを伝播します。

実行時に起きる順序

- ✓ 入力がresource ownershipの条件を満たすか確認する。
- ✓ owner SQLを実行し、途中の型と状態を追う。
- ✓ 404か403で失敗を成功経路から分離する。
- ✓ tenant isolationにより、出力と副作用を検証する。

型を声に出す 認可・IDOR・tenant境界に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つを変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 07 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

他人のid → rows_affected=0 → 404
自分のid → rows_affected=1

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 07 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[tokio::test] async fn cannot_access_other_users_note() {
    let a = seed_user().await;
    let b = seed_user().await;
    let note = seed_note(b.id).await;
    assert_eq!(show_as(a.id, note.id).await.status(), 404);
    assert_eq!(update_as(a.id, note.id).await.status(), 404);
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 07 · BACKEND APPLICATION

バックエンドのどこで使うか

管理者操作は暗黙の例外にせずRole/Permission型と監査ログを通します。tenant_idもすべてのqueryとunique indexへ含めます。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。

- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 07 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	resource ownershipに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	owner SQLの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	404か403またはtenant isolationの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 07・DEBUGGING

認可・IDOR・tenant境界を再現して切り分ける

認可・IDOR・tenant境界の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: resource ownership
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: owner SQL
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 404か403
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: tenant isolation

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	resource ownership	resource ownership — 入力と前提を確認し、境界を曖昧にしない。
2	owner SQL	owner SQL — 値がどの順で変換されるかを追跡する。
3	404か403	404か403 — 失敗を再現し、型またはログから原因を特定する。
4	tenant isolation	tenant isolation — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 認可はUIではなくresource queryへowner/tenant条件として埋め込みます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 07 · PRACTICE

手を動かす演習

二人のuserを作り、user Aがuser Bのnoteをshow/update/deleteできないintegration testを書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check  
cargo clippy -- -D warnings  
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 07 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[tokio::test]async fn cannot_access_other_users_note(){let a=seed_user().await;let b=seed_user().await;let note=seed_note(b.id).await;assert_eq!(show_as(a.id,note.id).await.status(),404);assert_eq!(update_as(a.id,note.id).await.status(),404);}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 認可はUIではなくresource queryへowner/tenant条件として埋め込みます。
- ✓ 認可・IDOR・tenant境界とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 08

Better Auth比較と推奨構成

UNIT 08 · CONCEPT

Better Auth比較と推奨構成

Better AuthはTypeScriptライブラリで、libSQL対応のrelational adapterを持ちますが、Topcoat内へ直接組み込む native Rust crateではありません。導入にはTS auth serviceとcookie/domain契約が増えます。

この単元の到達点

- TypeScript sidecar
- libSQL adapter
- cookie共有
- 運用境界の増加

なぜバックエンドで必要か

初心者向けRust教材では一プロセスのTopcoat Session + Argon2idが責務を追やすい選択です。Better Authを使う場合は認証 service境界として別章で設計します。



UNIT 08 · MENTAL MODEL

Better Auth比較と推奨構成を図でつかむ



Better AuthはTypeScriptライブラリで、libSQL対応のrelational adapterを持ちますが、Topcoat内へ直接組み込むnative Rust crateではありません。導入にはTS auth serviceとcookie/domain契約が増えます。図では左から右へ処理を追います。各箱の入口では入力 of 型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	TypeScript sidecar	TypeScript sidecar — 入力と前提を確認し、境界を曖昧にしない。
2	libSQL adapter	libSQL adapter — 値がどの順で変換されるかを追跡する。
3	cookie共有	cookie共有 — 失敗を再現し、型またはログから原因を特定する。
4	運用境界の増加	運用境界の増加 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出来されるか」を自分の言葉で説明します。

30秒説明

Better Auth比較と推奨構成とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 08 · MINIMUM CODE まず動く最小コード

Better Auth比較と推奨構成だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
// 構成A: Rust単体（本教材の推奨）
Browser -> Topcoat Session + Argon2id -> Turso
// 構成B: Better Auth sidecar
Browser -> TypeScript Better Auth -> shared session DB/JWT -> Rust Topcoat
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順で進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 08 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
Browser -> Topcoat Session + Argon2id -> Turso	この行が受け取る値、作る値、副作用の有無を確認します。
Browser -> TypeScript Better Auth -> shared session DB/JWT -> Rust Topcoat	この行が受け取る値、作る値、副作用の有無を確認します。
Better Auth比較と推奨構成	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がTypeScript sidecarの条件を満たすか確認する。
- ✓ libSQL adapterを実行し、途中の型と状態を追う。
- ✓ cookie共有で失敗を成功経路から分離する。
- ✓ 運用境界の増加により、出力と副作用を検証する。

型を声に出す Better Auth比較と推奨構成に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 08 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

Better Auth比較と推奨構成

アーキテクチャ図でRust単体構成とTypeScript sidecar構成の通信・DB・cookie境界を比較します。

観察ポイント

- TypeScript sidecar
- libSQL adapter
- cookie共有

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 08・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

選ぶ: 既存TS team/豊富なprovider/plugin活用/認証service共通化
選ばない: Rust単体学習/email-password中心/運用を一つにしたい
追加: CORS-cookie-domain/secret rotation/service障害とversion管理

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける

初心者から実務へ

63

観点	最小例	実務版
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 08 · BACKEND APPLICATION

バックエンドのどこで使うか

初心者向けRust教材では一プロセスのTopcoat Session + Argon2idが責務を追いやすい選択です。Better Authを使う場合は認証service境界として別章で設計します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 08 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	TypeScript sidecarに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	libSQL adapterの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	cookie共有または運用境界の増加の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 08・DEBUGGING

Better Auth比較と推奨構成を再現して切り分ける

Better Auth比較と推奨構成の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: TypeScript sidecar
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: libSQL adapter
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: cookie共有
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 運用境界の増加

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	TypeScript sidecar	TypeScript sidecar — 入力と前提を確認し、境界を曖昧にしない。
2	libSQL adapter	libSQL adapter — 値がどの順で変換されるかを追跡する。
3	cookie共有	cookie共有 — 失敗を再現し、型またはログから原因を特定する。
4	運用境界の増加	運用境界の増加 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 Better Authは有力ですがnative Rust統合ではなくsidecarです。本教材はTopcoat Session + Argon2idを主経路にします。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 08 ・ PRACTICE

手を動かす演習

Better Authを選ぶ条件、選ばない条件、追加運用項目を各3つ書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 08 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

選ぶ: 既存TS team/豊富なprovider/plugin活用/認証service共通化

選ばない: Rust単体学習/email-password中心/運用を一つにしたい

追加: CORS-cookie-domain/secret rotation/service障害とversion管理

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ Better Authは有力ですがnative Rust統合ではなくsidecarです。本教材はTopcoat Session + Argon2idを主経路にします。
- ✓ Better Auth比較と推奨構成とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 09

OIDC・managed IdP・MFA・回復

UNIT 09 · CONCEPT

OIDC・managed IdP・MFA・回復

企業SSOやsocial login、MFA、account recoveryが主要要件なら、自作資格情報よりOIDC対応IdPを検討します。Authorization Code + PKCE、state、nonce、redirect URIを守ります。

この単元の到達点

- OIDC provider
- Authorization Code + PKCE
- state/nonce
- account linking

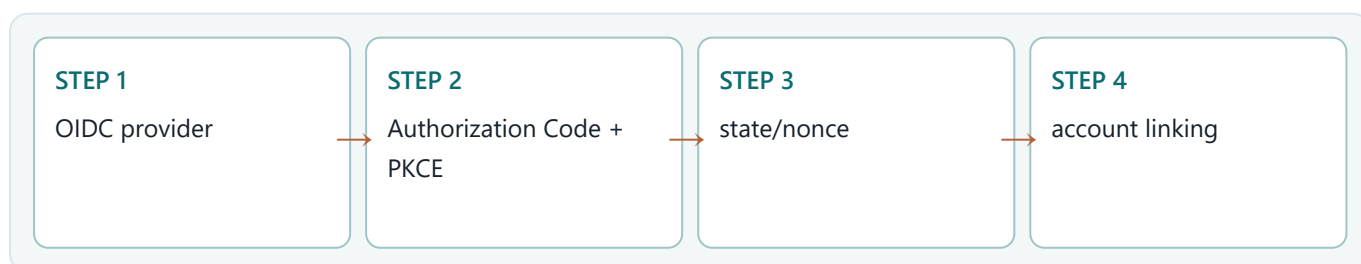
なぜバックエンドで必要か

provider emailだけでaccountを自動linkすると乗っ取りの危険があります。issuer+subjectを外部identityの主キーにし、linkには再認証を要求します。



UNIT 09 · MENTAL MODEL

OIDC・managed IdP・MFA・回復を図でつかむ



企業SSOやsocial login、MFA、account recoveryが主要要件なら、自作資格情報よりOIDC対応IdPを検討します。Authorization Code + PKCE、state、nonce、redirect URIを守ります。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口で

は結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	OIDC provider	OIDC provider — 入力と前提を確認し、境界を曖昧にしない。
2	Authorization Code + PKCE	Authorization Code + PKCE — 値がどの順で変換されるかを追跡する。
3	state/nonce	state/nonce — 失敗を再現し、型またはログから原因を特定する。
4	account linking	account linking — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

OIDC・managed IdP・MFA・回復とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 09 · MINIMUM CODE

まず動く最小コード

OIDC・managed IdP・MFA・回復だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
Browser -> /login -> IdP authorize
IdP -> /callback?code=...&state=...
Server: verify state, exchange code, validate issuer/audience/nonce
Server: map provider subject to local user, start local session
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 09 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
Browser -> /login -> IdP authorize	この行が受け取る値、作る値、副作用の有無を確認します。
IdP -> /callback?code=...&state=...	失敗時は早期returnし、呼び出し元へエラーを伝播します。
Server: verify state, exchange code, validate issuer/audience/nonce	この行が受け取る値、作る値、副作用の有無を確認します。
Server: map provider subject to local user, start local session	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力がOIDC providerの条件を満たすか確認する。
- ✓ Authorization Code + PKCEを実行し、途中の型と状態を追う。
- ✓ state/nonceで失敗を成功経路から分離する。
- ✓ account linkingにより、出力と副作用を検証する。

型を声に出す OIDC・managed IdP・MFA・回復に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 09・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
callback成功 → local session開始
state不一致 → 400 + 監査event
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 09・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
stateはsessionと一致
redirect URIは固定
token issuer/audience/expiry/signature
nonce一致
provider subjectを主キーにmapping
link/privilege変更時はsession rotate
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

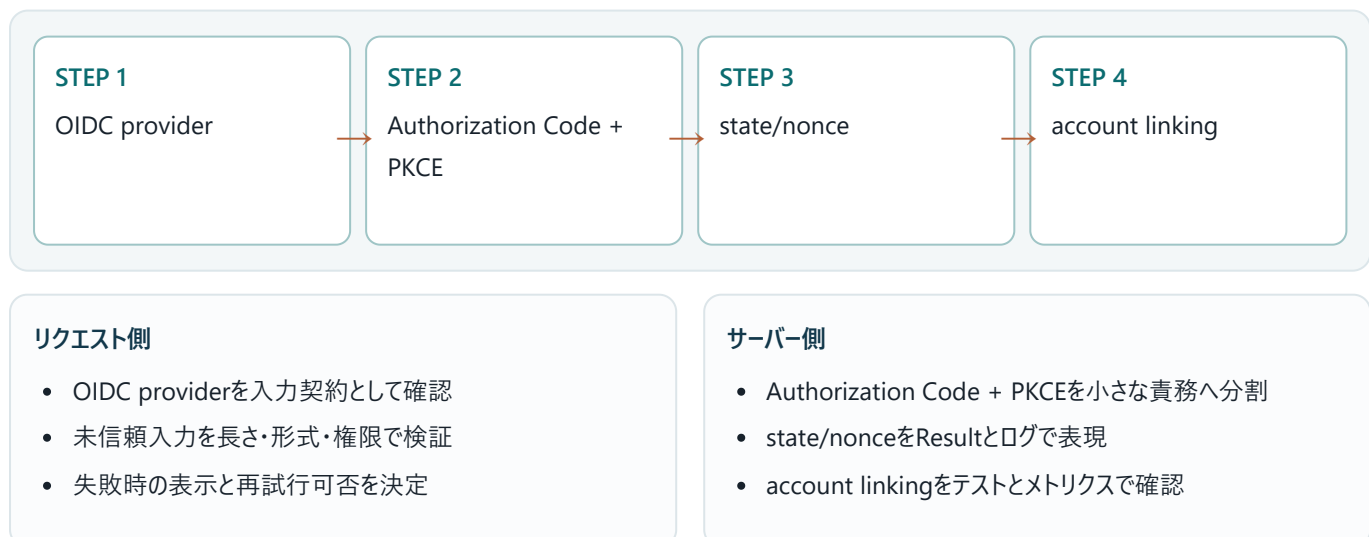
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 09 · BACKEND APPLICATION

バックエンドのどこで使うか

provider emailだけでaccountを自動linkすると乗っ取りの危険があります。issuer+subjectを外部identityの主キーにし、linkには再認証を要求します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 09 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	OIDC providerに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	Authorization Code + PKCEの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	state/nonceまたはaccount linkingの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 09・DEBUGGING

OIDC・managed IdP・MFA・回復を再現して切り分ける

OIDC・managed IdP・MFA・回復の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: OIDC provider
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: Authorization Code + PKCE
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: state/nonce
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: account linking

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	OIDC provider	OIDC provider — 入力と前提を確認し、境界を曖昧にしない。
2	Authorization Code + PKCE	Authorization Code + PKCE — 値がどの順で変換されるかを追跡する。
3	state/nonce	state/nonce — 失敗を再現し、型またはログから原因を特定する。
4	account linking	account linking — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 複雑な認証要件はIdPへ委譲しても、callback検証とlocal認可はアプリ責任です。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 09 ・ PRACTICE

手を動かす演習

OIDC callbackで必ず検証する項目をchecklistにしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 09 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
stateはsessionと一致
redirect URIは固定
token issuer/audience/expiry/signature
nonce一致
provider subjectを主キーにmapping
link/privilege変更時はsession rotate
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 複雑な認証要件はIdPへ委譲しても、callback検証とlocal認可はアプリ責任です。
- ✓ OIDC・managed IdP・MFA・回復とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 10

security test・監査・incident response

UNIT 10・CONCEPT

security test・監査・incident response

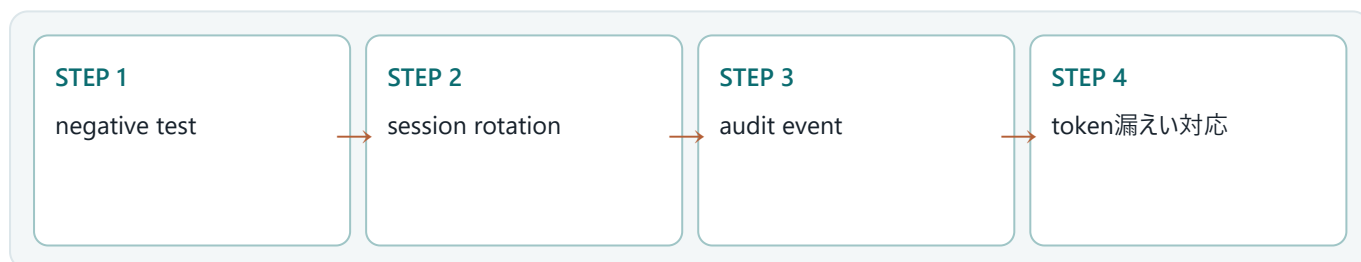
認証は正常loginだけでなく、列挙、CSRF、session fixation、期限、logout後、他人resource、rate limit、秘密漏えいを自動テストします。監査eventは誰が何をいつ試みたかを残します。

この単元の到達点

- negative test
- session rotation
- audit event
- token漏えい対応

なぜバックエンドで必要か

token漏えい時はsession revoke、secret rotate、影響期間のaudit検索、通知、再発防止を行います。password reset tokenもhash保存・単回使用・短期限にします。



先に答え

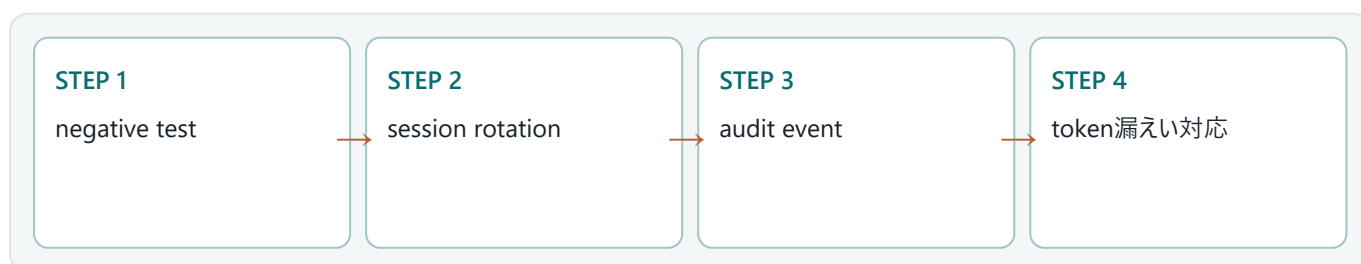
認証の品質は失敗系テスト、監査、revoke可能性、incident手順で測ります。

確認する問い

security test・監査・incident responseを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 10・MENTAL MODEL

security test・監査・incident responseを図でつかむ



認証は正常loginだけでなく、列挙、CSRF、session fixation、期限、logout後、他人resource、rate limit、秘密漏えいを自動テストします。監査eventは誰が何をいつ試みたかを残します。図では左から右へ処理を追います。各箱の入口では入力 of 型と信頼度、出口では結果

の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	negative test	negative test — 入力と前提を確認し、境界を曖昧にしない。
2	session rotation	session rotation — 値がどの順で変換されるかを追跡する。
3	audit event	audit event — 失敗を再現し、型またはログから原因を特定する。
4	token漏えい対応	token漏えい対応 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

security test・監査・incident responseとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 10・MINIMUM CODE

まず動く最小コード

security test・監査・incident responseだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[tokio::test]async fn logout_revokes_session(){let s=login().await;assert_eq!(account(&s).await.status(),200);logout(&s).await;assert_eq!(account(&s).await.status(),302);}#[tokio::test]async fn cross_user_update_is_denied(){/* seed two users and assert 404 */}
```

●●● 実行コマンド

```
cargo test auth security
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順で進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 10・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[tokio::test]async fn logout_revokes_session() {let s=login().await;assert</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>#[tokio::test]async fn cross_user_update_is_denied()/* seed two users and</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>security test</code> ・監査・incident response	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力negative testの条件を満たすか確認する。
- ✓ session rotationを実行し、途中の型と状態を追う。
- ✓ audit eventで失敗を成功経路から分離する。
- ✓ token漏えい対応により、出力と副作用を検証する。

型を声に出す security test・監査・incident responseに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 10・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

```
security tests ... ok
audit: login_failed reason=invalid_credentials account_hash=...
```

見える結果
端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと
型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 10・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

検知: 異常login/session利用をauditで確認
 封じ込め: 該当session全revoke/secret rotate
 根絶: 漏えい経路修正
 復旧: 再認証・監視強化
 事後: timeline/test/runbook更新

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

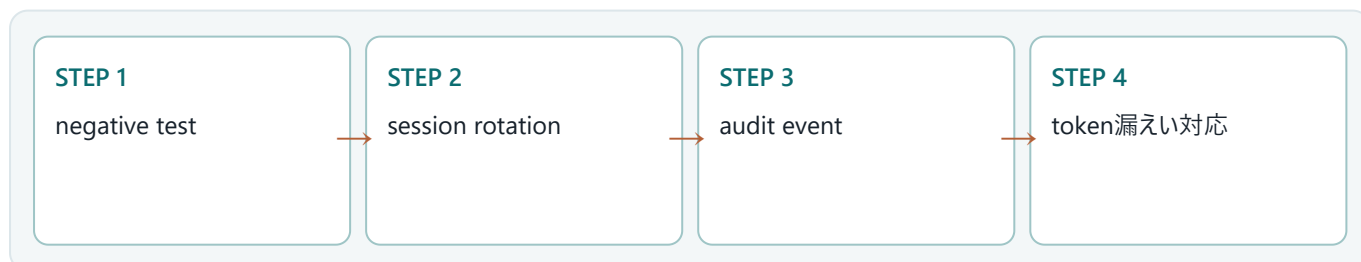
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 10 · BACKEND APPLICATION

バックエンドのどこで使うか

token漏えい時はsession revoke、secret rotate、影響期間のaudit検索、通知、再発防止を行います。password reset tokenもhash保存・単回使用・短期限にします。



リクエスト側

- negative testを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- session rotationを小さな責務へ分割
- audit eventをResultとログで表現
- token漏えい対応をテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 10 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	negative testに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	session rotationの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	audit eventまたはtoken漏えい対応の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 10・DEBUGGING

security test・監査・incident responseを再現して切り分ける

security test・監査・incident responseの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: negative test
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: session rotation
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: audit event
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: token漏えい対応

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	negative test	negative test — 入力と前提を確認し、境界を曖昧にしない。
2	session rotation	session rotation — 値がどの順で変換されるかを追跡する。
3	audit event	audit event — 失敗を再現し、型またはログから原因を特定する。
4	token漏えい対応	token漏えい対応 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 認証の品質は失敗系テスト、監査、revoke可能性、incident手順で測ります。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 10 ・ PRACTICE

手を動かす演習

認証incident runbookを検知、封じ込め、根絶、復旧、事後の五段階で書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 10 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

検知: 異常login/session利用をauditで確認

封じ込め: 該当session全revoke/secret rotate

根絶: 漏えい経路修正

復旧: 再認証・監視強化

事後: timeline/test/runbook更新

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 認証の品質は失敗系テスト、監査、revoke可能性、incident手順で測ります。
- ✓ security test・監査・incident responseとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

CAPSTONE

登録・login・logout・owner認可を備えた認証基盤を作る

10単元を一つの成果物へ統合します。動く結果だけでなく、途中のエラー、設計判断、確認コマンドも提出物です。

順	使う単元	統合する判断
01	認証・認可・session・脅威モデル	認証後も毎操作で認可し、曖昧な場合は拒否します。
02	パスワード・Argon2id・salt	平文ではなくsalt付きArgon2id PHC文字列を保存し、検証はpassword APIへ任せます。
03	登録・email正規化・重複・列挙対策	安価な検証→Argon2→UNIQUE INSERTの順に行い、競合を安全な表示へ変換します。
04	login・generic error・rate limit	外部エラーを一般化し、内部分類・rate limit・監視で攻撃と障害を区別します。
05	Topcoat session lifecycle	cookie token、DB hash、期限、userを一つのlifecycleとして更新します。
06	cookie・CSRF・Origin・SameSite	cookie属性、Origin検証、安全なHTTP methodを重ねてCSRFを防ぎます。
07	認可・IDOR・tenant境界	認可はUIではなくresource queryへowner/tenant条件として埋め込みます。
08	Better Auth比較と推奨構成	Better Authは有力ですがnative Rust統合ではなくsidecarです。本教材はTopcoat Session + Argon2idを主経路にします。
09	OIDC・managed IdP・MFA・回復	複雑な認証要件はIdPへ委譲しても、callback検証とlocal認可はアプリ責任です。
10	security test・監査・incident response	認証の品質は失敗系テスト、監査、revoke可能性、incident手順で測ります。

完了条件

- ✓ raw password/tokenを保存しない
- ✓ session期限とrevokeがDBで管理される
- ✓ cross-userとcross-originを拒否する
- ✓ Better Auth採否を要件表で説明できる

REVIEW

巻末確認問題

用語だけでなく、「小さな例」「バックエンドでの場所」「失敗時の挙動」を含めて教えてください。

1. 01. 認証・認可・session・脅威モデルを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
2. 02. パスワード・Argon2id・saltを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
3. 03. 登録・email正規化・重複・列挙対策を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
4. 04. login・generic error・rate limitを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
5. 05. Topcoat session lifecycleを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
6. 06. cookie・CSRF・Origin・SameSiteを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
7. 07. 認可・IDOR・tenant境界を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
8. 08. Better Auth比較と推奨構成を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
9. 09. OIDC・managed IdP・MFA・回復を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
10. 10. security test・監査・incident responseを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

答える順序 まず資料を閉じて説明し、次に最小コードを書き、最後に該当単元へ戻って不足を補います。正解の暗記ではなく、根拠を再現できることが目標です。

REVIEW ANSWERS

確認問題・解答の骨格

次の要点は模範解答の丸暗記用ではありません。自分の説明に「定義・小さな例・バックエンドでの場所」が含まれているかを照合します。

01. 認証・認可・session・脅威モデル

認証・認可・session・脅威モデルとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 認証後も毎操作で認可し、曖昧な場合は拒否します。

02. パスワード・Argon2id・salt

パスワード・Argon2id・saltとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 平文ではなくsalt付きArgon2id PHC文字列を保存し、検証はpassword APIへ任せます。

03. 登録・email正規化・重複・列挙対策

登録・email正規化・重複・列挙対策とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 安価な検証→Argon2→UNIQUE INSERTの順に行い、競合を安全な表示へ変換します。

04. login・generic error・rate limit

login・generic error・rate limitとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 外部エラーを一般化し、内部分類・rate limit・監視で攻撃と障害を区別します。

05. Topcoat session lifecycle

Topcoat session lifecycleとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: cookie token、DB hash、期限、userを一つのlifecycleとして更新します。

06. cookie・CSRF・Origin・SameSite

cookie・CSRF・Origin・SameSiteとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: cookie属性、Origin検証、安全なHTTP methodを重ねてCSRFを防ぎます。

07. 認可・IDOR・tenant境界

認可・IDOR・tenant境界とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 認可はUIではなくresource queryへowner/tenant条件として埋め込みます。

08. Better Auth比較と推奨構成

Better Auth比較と推奨構成とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: Better Authは有力ですがnative Rust統合ではなくsidecarです。本教材はTopcoat Session + Argon2idを主経路にします。

09. OIDC・managed IdP・MFA・回復

OIDC・managed IdP・MFA・回復とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 複雑な認証要件はIdPへ委譲しても、callback検証とlocal認可はアプリ責任です。

10. security test・監査・incident response

security test・監査・incident responseとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 認証の品質は失敗系テスト、監査、revoke可能性、incident手順で測ります。

GLOSSARY

重要用語と実務での位置

用語	一文定義	バックエンドでの位置
認証・認可・session・脅威モデル	認証後も毎操作で認可し、曖昧な場合は拒否します。	login済みだけでなく、routeごと・resourceごとにactorとownerを照合します。SQLのWHEREにもuser_idを含めます。
パスワード・Argon2id・salt	平文ではなくsalt付きArgon2id PHC文字列を保存し、検証はpassword APIへ任せます。	hash処理はspawn_blockingへ隔離し、同時数を制限します。parameterは負荷試験で調整し、将来はlogin成功時のrehashを検討します。
登録・email正規化・重複・列挙対策	安価な検証→Argon2→UNIQUE INSERTの順に行い、競合を安全な表示へ変換します。	登録と同時にsessionを開始するか、email verification後に開始するかを要件で決めます。重いhashより前に安価な形式検証を行います。
login・generic error・rate limit	外部エラーを一般化し、内部分類・rate limit・監視で攻撃と障害を区別します。	固定lockoutは攻撃者が他人を締め出せるため、遅延、短期bucket、CAPTCHA/step-up、通知を組み合わせます。
Topcoat session lifecycle	cookie token、DB hash、期限、userを一つのlifecycleとして更新します。	privilege変更ではrotateし、古いhash削除と新hash追加をtransactionで行います。全端末logoutはuser_idの全session行を削除します。
cookie・CSRF・Origin・SameSite	cookie属性、Origin検証、安全なHTTP methodを重ねてCSRFを防ぎます。	dangerous_disable_origin_verificationは独自CSRF対策を完全実装した場合以外使いません。API token clientとbrowser cookie clientの脅威を分けます。
認可・IDOR・tenant境界	認可はUIではなくresource queryへowner/tenant条件として埋め込みます。	管理者操作は暗黙の例外にせずRole/Permission型と監査ログを通します。tenant_idもすべてのqueryとunique indexへ含めます。
Better Auth比較と推奨構成	Better Authは有力ですがnative Rust統合ではなくsidecarです。本教材はTopcoat Session + Argon2idを主経路にします。	初心者向けRust教材では一プロセスのTopcoat Session + Argon2idが責務を追いやすい選択です。Better Authを使う場合は認証service境界として別章で設計します。
OIDC・managed IdP・MFA・回復	複雑な認証要件はIdPへ委譲しても、callback検証とlocal認可はアプリ責任です。	provider emailだけでaccountを自動linkすると乗っ取りの危険があります。issuer+subjectを外部identityの主キーにし、linkには再認証を要求します。
security test・監査・incident response	認証の品質は失敗系テスト、監査、revoke可能性、incident手順で測ります。	token漏えい時はsession revoke、secret rotate、影響期間のaudit検索、通知、再発防止を行います。password reset tokenもhash保存・単回使用・短期限にします。

PRIMARY SOURCES

公式資料と更新確認

仕様は更新されます。教材で意図を理解した後、実装時点の一次資料でAPI、security note、breaking changeを確認してください。

1. Rust Book

<https://doc.rust-lang.org/stable/book/>
Rust 2024 Edition を前提にした公式入門書

2. Rust Reference

<https://doc.rust-lang.org/reference/>
言語仕様を確認する一次資料

3. Standard Library

<https://doc.rust-lang.org/std/>
標準ライブラリAPI

4. Cargo Book

<https://doc.rust-lang.org/cargo/>
ビルド、依存関係、workspace、features

5. Rustdoc Book

<https://doc.rust-lang.org/rustdoc/>
APIドキュメントの書き方

6. Clippy

<https://doc.rust-lang.org/clippy/>
Rust公式Lint

7. Async Book

<https://rust-lang.github.io/async-book/>
非同期Rustの公式解説

8. Tokio Tutorial

<https://tokio.rs/tokio/tutorial>
Tokio公式チュートリアル

9. Topcoat

<https://github.com/tokio-rs/topcoat>
Topcoat公式リポジトリ。early-stageの注意を含む

10. Topcoat 0.5.0 API

<https://docs.rs/topcoat/0.5.0/topcoat/>
本教材で固定するAPI

11. Turso Rust SDK

<https://docs.turso.tech/sdk/rust/quickstart>
libSQL Rust SDK公式手順

12. Better Auth Database

<https://better-auth.com/docs/concepts/database>
Better AuthのDB構成

13. Fly.io Deploy

<https://fly.io/docs/launch/deploy/>
fly launch / fly deploy公式手順

14. Fly.io Health Checks

<https://fly.io/docs/reference/health-checks/>
公開後のヘルスチェック

15. OWASP Cheat Sheets

<https://cheatsheetseries.owasp.org/>
Webセキュリティの実務チェック

採用判断 本教材はRust 2024 Edition、Topcoat 0.5.0固定、Turso/libSQL、Topcoat Session + Argon2id、Fly.ioを主経路にします。Better AuthはTypeScript sidecarが適切な要件で比較します。